

Covert Channels in the `io_uring` Runtime

Joshua Gillett
Harvard University

Abstract

Linux’s `io_uring` subsystem is an asynchronous interface used to bypass the context-switching overhead of I/O system calls. `io_uring` uses shared-memory ring buffers between the user and the kernel to provide performant I/O operations. However, this still entails some amount of syscalls. In order to push a submission queue to the kernel, a participant process must call `io_uring_enter()`. If, as an ambitious systems programmer, you want to avoid this, `io_uring` exposes the option for a dedicated polling kthread (`SQPOLL`) that drains submissions without entering the calling task. Prior side-channel work on `io_uring` has focused on the contention against the storage device or the PCIe bus. We instead ask whether the runtime itself—its scheduler, registered-resource tables, and documented cross-ring primitives—can be used to communicate between two processes that share no permitted IPC.

We answer this question in the affirmative with three different layers of required capability for the attacker. (1) When two rings can share an `SQPOLL` backend via `IORING_SETUP_ATTACH_WQ`, a single shared kernel polling loop is enough to carry ≈ 86 bits/sec at a five-fold cross-validated bit error rate (BER) of 0.09% on a commodity laptop. We attribute this signal directly to the `SQPOLL` inner-loop scan via a Cohen’s- d versus ring-fan-in sweep. (2) When the rings cannot share an `SQPOLL` backend, but both call `IORING_REGISTER_FILES` on a common backing `struct file` (e.g., `/dev/null`), a sender can cause contention in `IORING_REGISTER_FILES_UPDATE` by modulating the `f_ref.refcnt` cache line, for which a receiver can load on every `io_file_get_fixed` call. Repetition coding allows this channel to transmit ≈ 0.625 bits/sec with a BER of 0.08. (3) When neither shared `SQPOLL` nor common registered resources are available, but the sender is able to hold a file descriptor to the receiver’s ring, `IORING_OP_MSG_RING` provides a documented and

kernel-approved cross-ring CQE delivery primitive that achieves a BER of 0.008 at ≈ 18 bits/sec.

1 Introduction

Compared to `epoll/aio`, `io_uring` removes a per-operation syscall trap by sharing two ring buffers between the kernel and a process, a Submission Queue (SQ) and a Completion Queue (CQ) [1]. With `IORING_SETUP_SQPOLL`, the kernel additionally spawns a dedicated kthread to poll the SQ and write the CQ on the user’s behalf, eliminating syscall traps on the hot path entirely. This design has made `io_uring` popular in high performance database engines, network proxies, and workloads that are generally bottlenecked by syscall overhead, and is increasingly enabled in multi-tenant cloud environments where co-resident, mutually distrustful processes share the same kernel [8, 10].

Use in said co-resident environments raises the security question, that has to our knowledge, not been answered for the runtime layer of `io_uring`. There is prior work, recently, on covert channels through the storage side of `io_uring`—Juffinger et al. demonstrate a covert channel that exploits PCIe-bus contention provoked by unprivileged `io_uring` I/O— but if a covert channel can be constructed by modulation in the runtime, storage-class mitigations for this covert channel cannot prevent disallowed communication [9].

Our project, thus, asks if two isolated processes can communicate covertly through `io_uring`’s runtime without shared user-space memory, IPC, and without touching the block-layer storage. The answer turns out to be that it is largely dependent on the system administrator’s sandboxing policy since different elements, and that the possibility mostly remains under a restrictive policy, but degrades. We, therefore, organize our results into three layers of attacker capabilities.

Channel I: shared SQPOLL backend. If both processes are able to create rings with `IORING_SETUP_ATTACH_WQ`, they will share a single SQPOLL kthread. The kthread’s hot loop will then iterate over every attached ring and process its SQ tail. A process can thus send a burst of `IORING_OP_NOP` operations on one ring to measurably extend the period between visits to the other process’s SQ. We obtain ≈ 86 bits/sec at a 5-fold cross-validated BER of 0.09% with $N = 2$ attached sender rings, a 4-ms symbol window, and a Fisher LDA decoder over five per-window features. We confirm that the channel is internal to SQPOLL by sweeping the number of attached rings $N \in \{1, 2, 4, 6, 8\}$ and measuring the magnitude of the Cohen’s d on the receiver’s per-window count of issued NOPs and find that it increases monotonically from -8.30 to -16.97 before saturating at $N \geq 6$ and giving Pearson $r = -0.53$ with permutation $p = 0.043$.

Channel II: registered-files cache-line bouncing. Removing `ATTACH_WQ` collapses Channel I across testing. The exception is in a registered-files harness in which the sender churns its fixed-file table via `IORING_REGISTER_FILES_UPDATE`. A kernel-source audit found that the per-struct `file` atomic refcount line (`f_ref.refcnt`) as the only candidate carrier that is touched on both sides of communication. To turn this audit into an empirical claim at covert communication, we ran a repetition-coded K -sweep at $K \in \{1, 2, 4, 8, 16, 32\}$ with a 50-ms symbol window. The receiver reaches a mean soft BER of 0.080 at $K = 32$ and has zero errors on 32 super-symbols in the best of three trials, but has throughput of 0.625 bits/sec.

Channel III: `IORING_OP_MSG_RING` CQE delivery `IORING_OP_MSG_RING` is a documented opcode that lets a sender ring post a CQE on a target ring that it holds a file descriptor for. With independent SQPOLL kthreads and no registered resources, modulating the rate of inbound `MSG_RING` deliveries on the receiver’s CQ ring perturbs its NOP-timing loop enough that the LDA decoder can decipher a message trivially. We measure $K = 1$ BER ≈ 0.008 at ≈ 18 bits/sec across three trials. The attacker’s only required capability beyond being able to setup `io_uring` is possession of a file descriptor to the victim’s ring which is plausible via the common ancestor process `SCM_RIGHTS` or through some other form of communication.

The rest of the paper is organized as follows. We first review the `io_uring` kernel mechanisms our channels exploit, state our threat model, and describe the common methodology of the channels. We then present the three channels in increasing order of sandbox restrictiveness. Finally, we discuss mitigations, related works, limitations, and future work.

2 Background

2.1 The `io_uring` interface

`io_uring` is the third-generation of Linux asynchronous I/O interfaces, introduced by Jens Axboe in 2019 [1]. A user process creates a ring with `io_uring_setup(2)`, which returns a file descriptor whose `mmap` regions back two queues: the SQ into which the user can write Submission Queue Entries (SQEs), and the CQ, on which the kernel posts Completion Queue Entries (CQEs). On a default ring, each SQE is delivered to the kernel via an `io_uring_setup(2)` syscall.

The runtime also supports a number of optional setup flags that change how, when, and by whom the SQ is drained. The two relevant to this work are as follows.

- `IORING_SETUP_SQPOLL`, which spawns a kernel polling thread to watch the SQ tail and submit work itself, so user code never has to call `io_uring_enter` on the hot path. By default the kernel spawns one such kthread per ring with this flag.
- `IORING_SETUP_ATTACH_WQ`, which causes a new ring to be attached to an existing ring’s SQPOLL backend rather than spawning its own. All rings sharing a backend are placed within a `io_sq_data` list and are drained by one kthread.

`io_uring` additionally exposes a registered-resources mechanism. `io_uring_register(2)` with `IORING_REGISTER_FILES` pins an array of file descriptors into a per-ring fixed-file table, after which an SQE can refer to a slot by index rather than by file descriptor to avoid `fget` and `fput` latency. `IORING_REGISTER_FILES_UPDATE` swaps the individual slots without re-registering the entire table. A symmetric mechanism exists for buffers in `IORING_REGISTER_BUFFERS`. Finally, the cross-ring opcode `IORING_OP_MSG_RING` lets one ring post a CQE on a target ring whose file descriptor it possesses [2].

2.2 Attack surface

The standard IPC protocol for Linux— pipes, sockets, signals, etcetera— is common for a security policy in a multi-tenant environment to account for. `io_uring`’s runtime introduces several new mediators that do not necessarily come to mind for a sandbox policy writer: a shared kernel polling thread, a per-ring registered-resource table whose slots may alias to same backing struct `file`, and a documented ring messaging opcode. Each has a kernel-side mechanism by which one ring’s behavior can be observed by another ring without invoking IPC.

Prior side-channel work has examined `io_uring` from the storage device or the PCIe interconnect, but the runtime also holds distinctive design [9]. Our work treats the runtime layer as a first-class attack surface and asks how much bandwidth and how clean a BER each analyzed runtime feature can carry under realistic, non-cooperative assumptions on shared state.

2.3 Covert timing channels and capacity

Lampson’s confinement-problem framing of covert channels treats them as any communication mechanism not explicitly intended for communication [12]. Our channels fit this strictly since the sender modulates a kernel-mediated quantity—a scan-loop visit period, a refcount cache-line state, or a CQ-arrival rate—that the receiver can observe through timing.

For analysis, we model each channel as a binary symmetric channel (BSC) whose crossover probability is the per-window BER p . The Shannon capacity is,

$$C(p) = 1 - H_2(p) \\ H_2(p) = -p \log_2 p - (1 - p) \log_2 (1 - p)$$

in bits per channel use. Any non-degenerate $p < \frac{1}{2}$ gives $C(p) > 0$, so the channel is information-theoretically useable regardless of how poor the per-symbol decision is. We exploit this explicitly with repetition coding: a length- K repetition code transmits each information bit K times at a rate of $1/K$, and hard-decision majority voting reduces the BER towards zero exponentially in K by a Chernoff bound. For $p = 0.367$, the $K = 1$ BER of our second channel, the analytic hard-majority p_K satisfies $p_4 \approx 0.34$, $p_{16} \approx 0.13$, $p_{32} = 0.058$, $p_{128} \approx 2.8 \times 10^{-3}$. Repetition is far from capacity-approaching since the $C(0.367) \approx 0.052$ ceiling at our 50-ms window corresponds to ≈ 1.04 info-bits/sec, an order of magnitude above what we deliver at $K = 32$, but it is the simplest possible code and serves our existence claim. We discuss capacity-approaching follow-ups in Section X.X.

3 Threat Model

We consider two co-resident mutually distrustful processes $\mathcal{S}$ (sender) and $\mathcal{R}$ (receiver) on a single Linux host. Both can call `io_uring_setup(2)` and submit SQEs, but neither has any sanctioned IPC channel to the other. The adversary controls $\mathcal{S}$ and seeks to leak data to a co-resident $\mathcal{R}$, which is also controlled by the adversary, without being recorded by some sandbox policy as performing IPC.

For each channel, we state explicitly what additional state, if any, $\mathcal{S}$ and $\mathcal{R}$ must arrange to share at ring-creation time. Whether each precondition is realistic de-

pends on the sandbox policy—e.g. `ATTACH_WQ` is plausible inside a single container’s `io_uring` pool but can be denied by `seccomp`, sharing a `struct file` is given for any globally shared inode (`/dev/null` is particularly convenient), but can be disallowed by mount or namespace separation, a passed ring file descriptor is plausible across descendants of a common process or via `SCM_RIGHTS` but can be revoked by sandbox API decisions.

We notably do not consider hardware-level side channels such as cache prime+probe [16, 20, 14], branch-predictor channels [5], or the particularly famous speculative-execution channels [11, 13], and instead focus our threat model on the runtime of `io_uring`. Where our channel interacts with hardware, we note this interaction explicitly, as in Section 6.4.

4 Methodology

This section describes the parts of the experimental design that are common to all three channels: how the sender modulates a binary signal, how the receiver instruments per-window features, how they are decoded, and how observed effects are attributed back to specific kernel mechanisms.

4.1 Symbol modulation

The sender encodes one bit per symbol window of width W milliseconds. In bit-1 windows, it “hammers” a configurable kernel operation (NOP storm, registered-file update churn, or `MSG_RING` flood) at a configured rate and in bit-0 windows it goes idle. The receiver runs its own probe loop at the same window width with epoch-aligned start times derived from `CLOCK_MONOTONIC`. Both sides emit per-window CSV records keyed by window index. A short payload prefix (“10101001110”) is our canonical 11-bit pattern for the `ATTACH_WQ` channel and longer pseudo-random patterns are used for the repetition-coded channel) repeats `LOOPS` times to amortise the per-trial sync cost.

4.2 Receiver instrumentation

The receiver issues its own `IORING_OP_NOP` (or, for Channel II, `IORING_OP_NOP` with the correct `nop_flags`, or for the buffer-register variant of Section X.X, `IORING_OP_READ_FIXED`) and times each submit/reap iteration with `rdtsc`. For each window, the receiver records:

- `samples`, the number of complete probe iterations.
- `median_cycles`, `p90_cycles`, `p99_cycles`,
- `mean_cycles`,

- `max_cycles`.

These features are saved to a CSV which the offline analyzer can read.

4.3 Decoder: Fisher LDA

Originally, a single feature and fixed-threshold rule decoder was used, but this performs poorly on `median_cycles`. Thus, we moved to a Fisher Linear Discriminant-Analysis (LDA) over the five-dimensional feature vector [6]. LDA picks the direction $\mathbf{w} \propto \sum_w^{-1} (\bar{\mathbf{x}}_1 - \bar{\mathbf{x}}_0)$ that maximizes between-class variance over within-class variance, projects each window onto $\mathbf{w}$, and classifies by sign relative to the midpoint. To avoid optimistic bias on small payloads, we score every cell by 5-fold stratified cross-validation: five times we hold out a fifth of the windows, fit on the remaining four-fifths, and report the held-out BER. The LDA-CV-BER reported throughout the paper is the mean of these five held-out folds.

4.4 Effect size and significance tests

The headline number of any covert channel is its BER, but two additional summaries help us reason about why a channel works. We use Cohen’s d on each per-window feature [3],

$$d_f = \frac{\bar{x}_{f,1} - \bar{x}_{f,0}}{s_{p,f}},$$

where $s_{p,f}$ is the pooled standard deviation, as our headline effect-size metric. $|d| \geq 0.8$ is conventionally “large.” We report d on samples as the cleanest discriminator under ATTACH_WQ, and on per-feature panels for the attribution sweeps. Cell-level statistical significance is reported via two-tailed permutation tests on either Pearson r (for the N -th sweep) or paired-trial means.

4.5 Falsifiers for kernel attribution

Once a channel is shown to exist with low enough BER to be useful, the next question we ask is which kernel mechanism carries said bits. Attribution is harder than measurement because the kernel’s hot path runs many primitives concurrently and the candidate carriers (a mutex, a per-task spinlock, an atomic refcount, an allocator partial-list lock) cannot be turned off individually for a given hot path. Our methodology, applied explicitly in Section X.X and X.X, is as follows.

- Read the kernel source (mirrored under `deps/kernel_io_uring/`, all citations against `torvalds/linux v6.13`), and enumerate every synchronization primitive that either side of the channel touches.

- For each candidate primitive P , design a binary in which the sender’s hot churn is the smallest possible workload that would amplify P if P were the carrier (e.g. `dup/close` for `current->files->file_lock` and `getpid()` for the bare-syscall noise floor).
- Run all variants under the same receiver, payload schedule, CPU pinning, and decoder, and record their BER under repetition coding.
- Tabulate. The variant whose BER is best after holding the receiver fixed points to the carrier, by contrast with the worst variant ruling out an alternative carrier which was not amplified.

4.6 Hardware environment

All experiments reported in this paper run on a single Fedora 42 laptop with an Intel Core i7-8650U (8 logical cores, single NUMA node, 1.9 GHz base clock), kernel 6.19. We pin the sender thread, receiver thread and SQPOLL kthread to distinct logical cores via `sched_setaffinity` and `IORING_SETUP_SQ_AFF`. We also use a vendored `liburing` build under `deps/liburing-install` so that the binary ABI is stable across runs. While the hardware is admittedly modest, we expect that a industry-scale server to have cleanly higher absolute capacities and lower BERs. We cite the implication of the limitation explicitly in Section X.

5 Channel I: Shared SQPOLL Backend

5.1 Mechanism

When a ring is created with `IORING_SETUP_ATTACH_WQ`, it joins an existing ring’s `io_sq_data` list. Inside `io_sq_thread` (`io_uring/sqpoll.c`) the polling kthread runs a round-robin loop:

```
list_for_each_entry(ctx, &sqd->ctx_list, ...)
    __io_sq_thread(ctx, ...);
```

At each visit to a context, the kthread reads the SQ tail, copies SQE bytes from the user-mapped ring into kernel state, dispatches each SQE, and posts CQEs. The wall-clock time spent inside one context’s visit directly delays the next visit to every other context on the same list. From the receiver’s point of view, the period between successive visits to its own ring is the per-NOP service rate.

5.2 Implementation details

We implement this channel as the binary, `attached_multi`, that takes `--num-sender-rings`

Table 1: Sub-10 ms symbol sweep at $N = 2$, $I = 100$ on `attached_multi`. “LDA CV BER” is the mean $\pm$ std across all five folds of all n trials. “#0” counts trials whose 5-fold CV BER was identically zero out of n .

W (ms)	n	$ d $ on samples	LDA CV BER	bits/s	#0
4	8	14.4 ± 2.3	0.0009 ± 0.0026	85.9	7
6	8	11.2 ± 1.7	0.0028 ± 0.0039	73.3	5
8	3	12.3 ± 2.1	0.0026 ± 0.0044	64.0	1
10	3	11.8 ± 5.2	0.0026 ± 0.0044	56.7	1
12	3	17.3 ± 3.6	0.0000 ± 0.0000	50.9	3
15	3	5.95 ± 2.8	0.0304 ± 0.0305	44.2	0

N as an argument. A base ring is created with `IORING_SETUP_SQPOLL` and `IORING_SETUP_SQ_AFF` so that the `kthread` is pinned to a fixed logical core. The sender then creates N additional rings with `IORING_SETUP_ATTACH_WQ` pointing to the base ring’s file descriptor, and the receiver creates one such ring in the same way. All $N + 2$ rings are on the same `io_sq_data` list and therefore share a single `kthread`. In bit-1 windows, each sender submits I in-flight `IORING_OP_NOP` SQEs, across its N rings. In bit-0 windows, it drains said SQ. The receiver’s probe loop is unchanged from the single-ring case.

5.3 Cell-grid sweep

We swept the Cartesian product $N \in \{1, 2, 4\} \times W \in \{10, 20, 30, 50\}$ ms $\times I \in \{100, 200\}$ SQEs per ring and recorded the LDA-CV-BER and wall-clock bits/sec for each triple. The Pareto-optimal cell—dominant on both axes—was $N = 2$, $W = 10$ ms, $I = 100$, at 56.7 bits/sec and LDA-CV-BER = 0.000. Below $W = 10$ ms, we expect that we hit the SQPOLL wakeup-jitter on the CPU and above $W = 50$, we simply lose throughput.

5.4 Sub-10 ms symbol sweep

To push this cell finer, we ran a sub-10 ms sweep with $W \in \{4, 6, 8, 10, 12, 15\}$ ms at $N = 2$, $I = 100$. Three trials at each cell exposes two candidates as superior ($W = 4$ and $W = 6$). We then, ran 8 trials at each. Table 1 summarizes the results of this.

The Pareto-optimal cell is, therefore, $W = 4$ ms, where the receiver manages enough probe iterations per window to separate the classes. Thus, we adopt ($N = 2$, $W = 4$ ms, $I = 100$) as the configuration for Channel I. Across 8 trials with 5-fold CV each, the configuration emits a single bit error on only one fold, for an aggregate LDA-CV-BER of 0.0009. Wall-clock throughput is 85.9 bits/sec including a 1-second sync window.

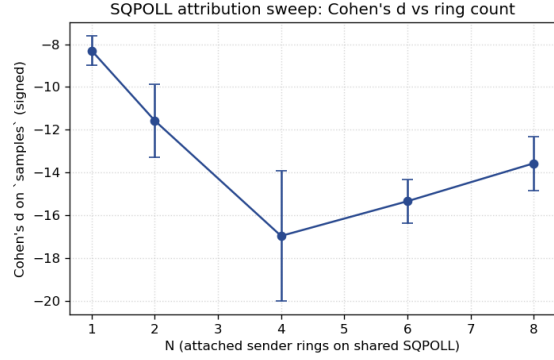


Figure 1: Cohen’s d on the receiver’s `samples` feature versus the number N of attached sender rings funnelled into one shared SQPOLL backend (mean $\pm$ std over 3 trials per cell). The aggregated correlation has Pearson $r = -0.532$, Spearman $\rho = -0.600$, and two-tailed permutation $p = 0.043$ ($n_{\text{perm}} = 10000$).

5.5 Attribution

The throughput gain and low BER are not yet evidence that the channel runs through the SQPOLL portion of the runtime, because adding rings increases both kernel-side load and CPU-side cache pressure. To attribute the signal directly to the `kthread`’s per-pool inner loop, we hold the operating point fixed at $W = 100$ ms and sweep $N \in \{1, 2, 4, 6, 8\}$ with three trials per cell.

Our hypothesis is as follows. If the carrier is the SQPOLL inner-loop scan, the magnitude of Cohen’s d on the receiver’s `samples` feature should change monotonically with N before saturating around the value of N at which the inner-loop scan itself becomes the bottleneck (rather than per-ring SQE processing). The competing hypothesis that the leak is L1/L2 cache or scheduler contention around `io_uring` but not inside it predicts a mostly flat or noisy curve.

Figure 1 shows the results of this experiment. The signed Cohen’s d on `samples` drops from -8.30 ± 0.69 at $N = 1$ to -16.97 ± 3.06 at $N = 4$ and then recovers to -13.57 ± 1.26 at $N = 8$. The saturation around $N = 4$ and recovery beyond are consistent with two effects. At $N \geq 4$, the inner-loop scan itself is the dominant cost (so adding rings no longer widens the gap), and at $N = 8$ the sender thread’s CFS scheduler across rings limits the effective per-ring submission rate. We did not push N further because of the limitations of the 8-logical-core CPU that was used.

The aggregated 15-cell statistic with Pearson $r = -0.532$, Spearman $\rho = -0.600$, two-tailed permutation $p = 0.043$ at $n_{\text{perm}} = 10000$ is significant at $\alpha = 0.05$. We read this as direct evidence that the carrier must be internal to the SQPOLL runtime, since the rings share no

other state in this experiment.

6 Channel II: Registered-Files Cache-Line Bouncing

A natural follow-up to the first channel is whether a covert channel in the `io_uring` runtime can exist without `ATTACH_WQ`. The simple version of this question has a clean answer in the negative, but finding the more interesting positive answer required a kernel-source audit and a five-binary falsifier table, which we present in this section.

6.1 Sweep without `ATTACH_WQ`

We first test the simple question with eight variants run at the same canonical cell of $W = 100\text{ms}$, $I = 200$, payload 10101001110, `LOOPS=12`, with 1 second syncs and in 5 trials each. Each variant differs only in what the sender modulates, and the receiver’s probe loop and LDA decoder remain unchanged from Section 5. Table 2 summarizes these results.

We note from this that CPU-sharing alone (independent_pair, same-CPU) and allocator pressure alone (many_ring_pressure, $N = 8$) were unable to produce a measurable signal. The only exception to our trials is that `regfile_pair`, our test with register-update on registered files achieves $|d| = 0.26 \pm 0.13$, which is $3\times$ the next-best variant. The signal is too small to decode but is well above the noise floor of every other variant, and so deserves a focused follow-up.

6.2 Kernel-source audit

From reading `io_uring/register.c`, `io_uring/rsrc.c`, `fs/file.c`, and `fs/file_table.c` at v6.13, we enumerate every lock and atomic on the call chain of the sender’s `io_uring_register_files_update` syscall and the receiver’s `IORING_OP_NOP` with `IOSQE_FIXED_FILE` set. Eleven primitives appear upon these paths. We list the following four with the strongest prima-facie case for being the carrier.

- (a) `ctx->uring_lock` is held by the sender for the duration of the registers-update syscall and by the receiver’s `SQPOLL` kthread during `SQE` submission. However, each ring has its own `ctx->uring_lock`. Thus, we rule it out by inspection.
- (b) `current->files->file_lock` is the per-task spinlock that guards file-descriptor table mutations. By inspection, it is taken on `dup`, `close`, and on `_alloc_fd` write paths. It is not taken on `fget`,

which uses RCU. Neither the sender’s register-update path nor the receiver’s `NOP` path takes `file_lock` in steady state, but this hypothesis is quite natural so we explicitly build a falsifier to test it.

- (c) `kmem_cache_node->list_lock` for the `kmalloc-32` cache which backs `struct io_rsrc_node` and the per-CPU `pcp_lock` for the page allocator. The sender’s `kzalloc(sizeof(io_rsrc_node))` inside `io_register_files` hits this slab and acquires the per-`NUMA` `list_lock`. The receiver’s `io_kiocrb` allocations come from a different `kmem_cache`, so direct `list_lock` is not possible, but both caches share the underlying page allocator. Heavy churn from the sender can, thus, drain the per-CPU `PCP` free list and force the next allocation on any CPU in the same `NUMA` zone to refill from the buddy allocator under `zone->lock`. This is, therefore, a candidate.
- (d) `file->f_ref.refcnt` is simply an `atomic64_t` inside `struct file` which is incremented by `fget` and decremented by `fput` through full-barrier `RMWs`. When all dups in our churn pool alias to the same `struct file` for `/dev/null`— they do, by construction— every register-update produces two `RMWs` on the same cache line. If the receiver’s `NOP` path also reads `f_ref`, `MESI` coherence will bounce the line between the two CPUs. This is clearly also a candidate.

We expect (d) to be the strongest candidate since it is the only primitive on the audit whose backing memory is the same on both sides. We thus, also taking into account what we believe to be a small UX bug in `io_uring`, test these under repetition coding.

6.3 Repetition-coding empirical results

At $W = 50\text{ms}$, `regfile_nop` (the same test as `regfile_pair` with churn rate $1/64$, but with the UX fix) achieves a per-window BER of $p \approx 0.367$ which is well below 0.5 but still above what should be considered usable. We thus apply repetition coding. Each information bit is transmitted as K contiguous copies, and the receiver builds non-overlapping super-symbols of K windows and decodes by majority vote. We sweep $K \in \{1, 2, 4, 8, 16, 32\}$ for the five binaries in our falsifier set.

Figure 2 plots K -vs-BER curves and Table 3 reports the underlying numbers for each of the five binaries in our falsifier set. The `regfile_nop` BER falls $4.6\times$ from 0.367 at $K = 1$ to 0.080 at $K = 32$. One of the three trials

Table 2: No-ATTACH_WQ sweep with 5 trials each. `attached_pair_ctrl` retains ATTACH_WQ as a positive control.

Variant	n	$ d $ samples	$ d $ p99	uni-test BER	LDA CV BER	Verdict
<code>attached_pair_ctrl</code> (with ATTACH_WQ)	5	8.73 ± 1.14	2.48 ± 1.25	0.024 ± 0.018	0.009 ± 0.003	strong (control)
<code>independent_pair</code> , different-CPU SQPOLL	5	0.06 ± 0.07	0.13 ± 0.15	0.455 ± 0.039	0.455 ± 0.044	no channel
<code>independent_pair</code> , same-CPU SQPOLL	5	0.08 ± 0.11	0.08 ± 0.07	0.519 ± 0.044	0.550 ± 0.040	no channel
<code>many_ring_pressure</code> ($N = 8$), different-CPU	5	0.10 ± 0.09	0.14 ± 0.07	0.481 ± 0.038	0.448 ± 0.035	no channel
<code>many_ring_pressure</code> ($N = 8$), same-CPU	5	0.08 ± 0.11	0.05 ± 0.14	0.501 ± 0.032	0.515 ± 0.044	no channel
<code>regfile_pair</code> , churn rate 1/64	5	0.26 ± 0.13	0.19 ± 0.17	0.343 ± 0.106	0.432 ± 0.052	marginal
<code>regfile_pair</code> , no update churn	5	0.14 ± 0.09	0.10 ± 0.10	0.424 ± 0.083	0.451 ± 0.068	no channel
<code>hotpath_pair</code> , default task_work	5	0.10 ± 0.06	0.08 ± 0.14	0.491 ± 0.050	0.510 ± 0.051	no channel

Table 3: Repetition-coding K -sweep across the five attribution binaries (mean of 3 trials, soft-decoder BER). Schedule: 16 information bits $\times$ MAX_K = 32 $\times$ L00PS = 2 = 1024 symbol windows of 50 ms at $I = 200$, sender CPU 1, receiver CPU 2, sender SQPOLL CPU 3, receiver SQPOLL CPU 4.

Variant	$K = 1$	$K = 2$	$K = 4$	$K = 8$	$K = 16$	$K = 32$
<code>regfile_nop</code> (corrected receiver)	0.367	0.360	0.346	0.265	0.141	0.080
<code>regfile_diffproc</code> (cross-process)	0.366	0.348	0.339	0.279	0.210	0.152
<code>regfile_pair</code> (legacy, broken receiver)	0.396	0.385	0.386	0.349	0.255	0.233
<code>getpid_storm_pair</code> (syscall noise floor)	0.409	0.401	0.401	0.354	0.285	0.231
<code>dup_close_pair</code> (file_lock fingerprint)	0.388	0.370	0.376	0.347	0.219	0.272

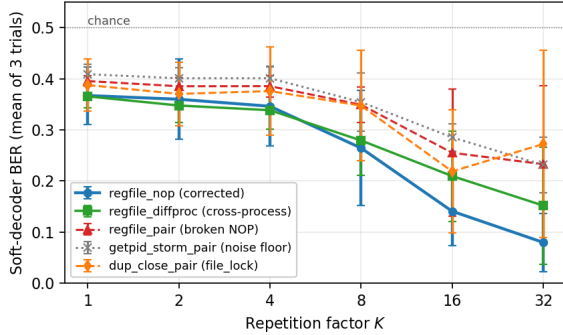


Figure 2: Repetition K -sweep across the five attribution binaries. Each point is the soft-decoder BER averaged over three trials of 1024 symbol windows

reaches zeros errors on 32 super-symbols. At $W = 50$ ms and $K = 32$ this has throughput 0.625 bits/sec.

6.4 Attribution

We can see from the falsifier data that ultimately the `refcnt` of the shared file descriptor to `/dev/null`. We note also that `regfile_diffproc`— which is the test in which the sender and receiver open separate kernel objects for `/dev/null`— performs well, reaching $K = 32$ soft BER of 0.152 which is below the noise floor of `getpid_storm_pair` but above that of `regfile_nop`. This is evidence of two facts. The dominant carrier in `regfile_nop` is the shared `struct file` of `/dev/null`.

This is evident since `regfile_nop` performs better than `regfile_diffproc`. Secondly, there exists a secondary carrier that survives when no `struct file` is shared. Cross-process, the only kernel resource the two rings provably share is global allocator state: the `kmalloc-32` partial-list locks, per-NUMA zone- $\rightarrow$ lock, and per-CPU `pcp_lock` of mechanism (c).

This is also where Channel II touches hardware most directly. The carrier of (d) is ultimately MESI traffic on a single cache line, but the sender is provoking this with only user-visible `io_uring` primitives. We treat this carefully since the magnitude of the signal is set by the cross-CPU coherence penalty of the underlying hardware.

7 Channel III: Cross-Ring CQE Delivery

We wonder here if it is possible for a covert channel to exist when neither ATTACH_WQ nor a shared `struct file` is available and the rings are isolated. It turns out that a documented opcode is apart of modern versions of `io_uring` with intended use for cross-ring asynchronous communication. We, thus, offer a channel, simply to say that a system administrator should prevent this opcode, `IORING_OP_MSG_RING`, on multi-tenant mutually distrustful systems.

7.1 Mechanism

`IORING_OP_MSG_RING` takes a target ring file descriptor and posts a CQE on that target ring [2]. Internally, `io_msg_ring_data` (in `io_uring/msg_ring.c`) ac-

quires the target context’s `completion_lock`, fills out a CQE, and wakes the target’s wait list if needed. From the target’s point of view, a CQE has appeared on its CQ ring without any SQE having been submitted itself.

The receiver’s per-window probe loop drives the channel. When the sender is storming MSG_RING at target `receiver_ring_fd`, we see three things happen. We first drain any inbound MSG_RING delivery CQEs already in the receiver’s CQ ring. We, then, allow for the receiver’s subsequent reap to return immediately because a fresh MSG_RING CQE is already there. It then consumes the CQE instead of waiting for its NOP CQE. Third, the iteration count per 50 ms window jumps from ≈ 12000 in bit-0 windows to ≈ 22000 in bit-1 windows. LDA on samples picks this out trivially.

7.2 Empirical results

We ran `msg_ring_pair` for three trials at $W = 50$ ms, $I = 200$, $LOOPS = 2$, $K = 1$, decoded with the same LDA pipeline as before. Per trial $K = 1$ hard BERS were 0.0077, 0.0000, and 0.0154, giving a mean of 0.008 ± 0.008 . We have a wall-clock throughput of ≈ 18.1 bits/sec at $K = 1$. The $K = 1$ BER alone is an order of magnitude lower than `regfile_nop` (≈ 0.367) and considerably better than the single-ring `attached_pair` cell at $W = 100$ ms (≈ 0.14).

7.3 Adjacent cross-ring opcodes

We also tested two adjacent cross-ring opcodes that do not go through MSG_MSG.

The first is `buffer_register_pair`, which streams `IORING_OP_READ_FIXED` of `/dev/null` plus `IORING_REGISTER_BUFFER_UPDATE` on the sender with the receiver also doing `READ_FIXED`. Its $K = 1$ BER is 0.422 ± 0.043 . The receiver’s `vfs_read` cost dominates, so it is largely insensitive to the buffer-resource churn.

The second is `openat_close_pair` which streams `IORING_OP_OPENAT` and `IORING_OP_CLOSE` on `/dev/null` on the sender, with the receiver simply doing `OP_NOPs`. Its $K = 1$ BER is 0.461 ± 0.014 , also indistinguishable from chance. This is simply because the receiver has no file descriptor touching operation in its hot path.

These indicate that a cross-ring opcode must have some ability to manifest as a quantity that the hot path observes.

8 Defenses and Mitigations

The defenses appropriate to each channel’s layer of attacker capability differ in both scope and in which maintainer would be responsible for them. We list four options

that are specific to the channel. We do not have full performance measurements for any of them, but each is within scope for future work.

Channel I: deny `ATTACH_WQ` across security domains. `IORING_SETUP_ATTACH_WQ` is the only flag at ring-creation time that opts into shared SQPoll backends. A seccomp-bpf or LSM policy can block this flag on `io_uring_setup` for processes outside of a single trust domain. This is likely the cheapest defense, since it only limits the efficiency of SQPoll performance for legitimate multi-ring applications. We claim that a per-cgroup or per-uid exemption is straightforward.

Channel II: prevent shared `struct file` across trust domains. Channel II is dominantly carried by the per-`struct file` atomic refcount cache line. The cleanest mitigation thus is to ensure that two rings in mutually distrustful sandboxes cannot register file descriptors that resolve to the same backing `struct file`. Mount-namespace separation already blocks the common case, but any LSM that mediates `IORING_REGISTER_FILES` can also enforce a uniqueness predicate. The residual cross-process signal we measure in `regfile_diffproc` is delivered by global allocator pressure and therefore is not addressed by this defense.

Channel II: per-context memory pool for `io_rsrc_node_alloc`. A patch to `io_uring/rsrc.c` could draw `io_rsrc_node` from a per-ctx mempool sized to `IO_NODE_ALLOC_CACHE_MAX` which the `io_uring` task-work infrastructure already does for adjacent small structs. This would remove the dependency on the global `kmalloc-32` partial list, limiting the `regfile_diffproc` residual signal.

Channel III: block `IORING_OP_MSG_RING` `IORING_OP_MSG_RING` is a documented cross-ring primitive that is, by design, to allow communication between rings. The defense is thus simply to disallow this at the policy layer by refusing to allow a MSG_RING SQE whose target ring’s owning task is in a different sandbox. As with the first mitigation, this can be implemented with eBPF or LSM [7].

9 Related Work

Storage-side covert channels in `io_uring`. The closest prior work is that found in *Secret Spilling Drive* [9], which finds that unprivileged `io_uring` can saturate the PCIe bus and provoke a measurable timing side channel in a co-resident victim’s SSD reads. Our work is complementary to this since their channel runs through the

storage device and ours runs through the runtime above the block layer.

Cache and CPU-internal side channels. FLUSH+RELOAD and the last-level cache attacks of Liu et al. [20, 14] directly module cache-line state between an attacker and a victim in a very similar manner to our Channel II attack. The carrier of both attacks is the MESI traffic of a cache line. In fact, work has been done to make use of this for covert communication with cross-VM caches [15, 17, 19]. Our work, of course, uses the `io_uring` user-visible primitives to carry out covert communications, while theirs relies, at least in some part, on the ability to issue `clflush` or to arrange cache-set conflicts.

Kernel-runtime monitoring. RINGGUARD [7] proposes an eBPF-based monitoring layer for `io_uring` that is aimed at detecting suspicious patterns of opcode use. This would be perfect our proposed mitigations, and it remains future work to implement it as such.

Information theoretics. The theory behind repetition coding and capacity that we describe in Section 2 is rather textbook [18, 4]. We direct interested readers towards more knowledgeable sources, and we make no claims of information-theoretic optimality. In fact, we only make the claim of existence.

10 Discussion

10.1 Limitations

We note that all experiments were ran on one Intel Core i7-8650U laptop. Replication on other forms of hardware remains future work. However, it is likely that hardware that is not as noisy as this machine should push SQPOLL’s noise floor down and create a better opportunity for Channel I. This hardware is also single-NUMA and a multi-socket Xeon to test the cross-NUMA also remains future work. We expect that it might reduce the residual capacity of the `regfile_diffproc` performance.

10.2 Future work

Our propositions of the most important future work endeavours is as follows.

Capacity-approaching codes. Replacing repetition with a soft-decision LDPC or polar code for Channel II could improve the throughput by drastic amounts. If this can be done, especially in regards to `regfile_diffproc`, the result would be most interesting.

Multi-Socket NUMA. If we pin the sender to socket 0 and the receiver to socket 1, we can test cross-NUMA performance for Channel II. We predict, as we said just earlier, that the cross-NUMA penalty might reduce the residual capacity and allow our mitigation to be better scoped.

An `io_rsrc_node` mempool patch. A mempool patch, as discussed in the mitigations, could close Channel II at the (c)-mechanism level.

10.3 Course goals achieved

We claim that we fully achieved our 100% goal of creating and verifying the existence of a covert channel in the runtime—in fact, we produced three of them. We also partially achieved the 125% goal of mitigation strategies by introducing some potential ideas, but we have not tested them, and leave that as future work.

11 Conclusion

We demonstrated that `io_uring`’s runtime has at least three distinct covert timing channels, and we organized them according to the amount of shared kernel state required of the cooperating adversarial processes. With `ATTACH_WQ`, a single shared SQPOLL kernel thread is enough to deliver a BER near zero at ≈ 86 bits/sec, and we attribute this directly to the kthread’s per-pool inner loop. With independent SQPOLL backends, but a shared backing `struct file`, the per-`struct file` atomic recount cache liner carries a $p = 0.37$ binary symmetric channel that repetition coding turns into a usable channel but at bits/sec rates at < 1 bits/sec. We also note a primitive that `io_uring` includes that allows for a very simple channel with `MSG_RING`.

We also provide potential avenues for mitigation such as restricting `ATTACH_WQ`, sandboxing file descriptor uniqueness, and disallowing `MSG_RING` for sensitive systems.

References

- [1] Jens Axboe. Efficient IO with `io_uring`. https://kernel.dk/io_uring.pdf, 2019. Accessed 2026.
- [2] Jens Axboe. [PATCH] `io_uring`: add support for `IORING_OP_MSG_RING` command. <https://lore.gnuweeb.org/io-uring/35b8fdf8-25c0-0aae-417c-c9f808a27643%40kernel.dk/T/>, 2022. Posted March 10, 2022.

- [3] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition, 1988.
- [4] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.
- [5] Dmitry Evtuyushkin, Ryan Riley, Nael Abu-Ghazaleh, and Dmitry Ponomarev. BranchScope: A new side-channel attack on directional branch predictor. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*, 2018.
- [6] Ronald A. Fisher. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936.
- [7] Wei He, Hangtian Lu, Fengwei Zhang, and Sen Wang. RingGuard: Guard io_uring with eBPF. In *Proceedings of the 1st Workshop on eBPF and Kernel Extensions*. ACM, 2023.
- [8] Maximilian Jasny, Muhammad El-Hindi, Tobias Ziegler, Viktor Leis, and Carsten Binnig. io_uring for high-performance DBMSs: When and how to use it. *arXiv preprint arXiv:2512.04859*, 2025.
- [9] Jonas Juffinger, Fabian Rauscher, and Daniel Gruss. Secret spilling drive: Leaking user behavior through SSD contention. In *Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2025.
- [10] Patrick Kalmbach, Walter Weiß, Christian Knapp, Anastasia Ailamaki, and Viktor Leis. What modern NVMe storage can do, and how to exploit it: High-performance i/o for high-performance storage engines. In *Proceedings of the VLDB Endowment*, 2023.
- [11] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. Spectre attacks: Exploiting speculative execution. In *2019 IEEE Symposium on Security and Privacy*, 2019.
- [12] Butler W. Lampson. A note on the confinement problem. *Communications of the ACM*, 16(10):613–615, 1973.
- [13] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Anders Fogh, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, and Mike Hamburg. Meltdown: Reading kernel memory from user space. In *27th USENIX Security Symposium*. USENIX Association, 2018.
- [14] Fangfei Liu, Yuval Yarom, Qian Ge, Gernot Heiser, and Ruby B. Lee. Last-level cache side-channel attacks are practical. In *2015 IEEE Symposium on Security and Privacy*, 2015.
- [15] Clémentine Maurice, Manuel Weber, Michael Schwarz, Lukas Giner, Daniel Gruss, Carlo Alberto Boano, Stefan Mangard, and Kay Römer. Hello from the other side: SSH over robust cache covert channels in the cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2017.
- [16] Colin Percival. Cache missing for fun and profit. In *BSDCan 2005*, 2005.
- [17] Thomas Ristenpart, Eran Tromer, Hovav Shacham, and Stefan Savage. Hey, you, get off of my cloud: Exploring information leakage in third-party compute clouds. In *Proceedings of the 16th ACM Conference on Computer and Communications Security (CCS)*, 2009.
- [18] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423, 623–656, 1948.
- [19] Zhenyu Wu, Zhang Xu, and Haining Wang. Whispers in the hyper-space: High-bandwidth and reliable covert channel attacks inside the cloud. In *21st USENIX Security Symposium*. USENIX Association, 2012.
- [20] Yuval Yarom and Katrina Falkner. FLUSH+RELOAD: A high resolution, low noise, L3 cache side-channel attack. In *23rd USENIX Security Symposium*. USENIX Association, 2014.